NeatAgent: A Local AI Agent for Autonomous File Organization

Abdulmohsen Alghannam
Columbia University
afa2165@columbia.edu

April 25, 2026

Synopsis

NeatAgent is a local AI agent that organizes files on your machine. You point it at a folder, it reads the contents of your files, groups them into meaningful subfolders, renames anything with a generic name, shows you the plan, and executes it after you approve. Nothing leaves your machine which means no cloud API, no subscription, no internet connection needed.

The agent is built on a **PLAN-AND-ACT** loop: it creates a complete plan before touching anything, then executes one step at a time. If a step fails, a Reflector node decides whether to retry, skip, or replan. The whole thing is wired together using **LangGraph**, a library for building graph-based agent workflows, and runs inference locally through **Ollama**, which serves open-weight language models on your own hardware.

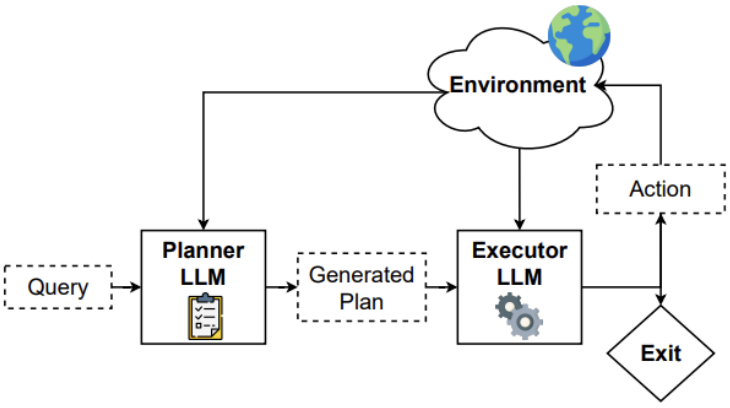

Figure 1: Overview of the Plan-and-Act framework, illustrating the separation between high-level planning and step-by-step execution

One thing that made the project interesting was figuring out how to get small local models to produce reliable, structured output. I went through a few iterations before landing on the current design, and a lot of the work ended up being about preventing the model from hallucinating files that don't exist. I'll explain how I solved that in the Novelty section.

For evaluation, I created a dataset of 10 files with generic names (file1.txt, file4.pdf, etc.) covering different content types such as invoices, health reports, gym logs, photos, and CSVs. The goal was to see whether the agent could classify files correctly based on *content* alone, since the filenames give no signal. I compared three models of different sizes: `qwen2.5-coder:14b`, `qwen3:8b`, and `llama3.2:3b`.

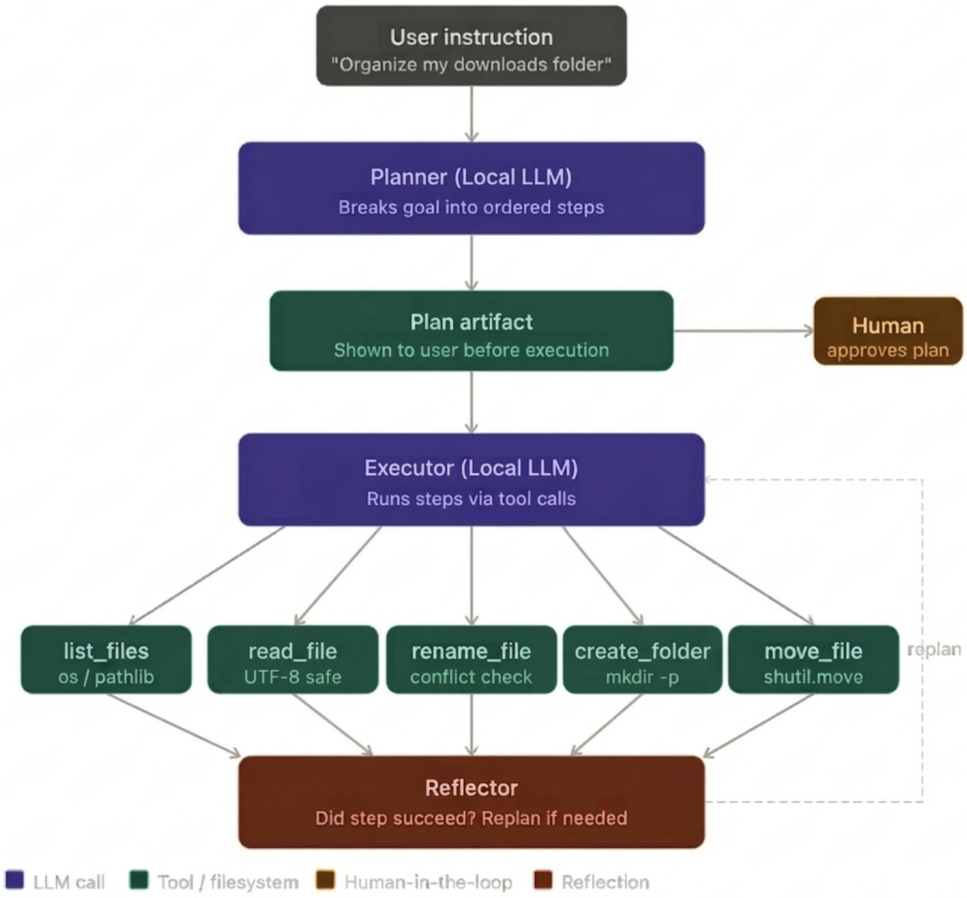

Figure 2: NeatAgent architecture – task is decomposed into a planner phase which creates a plan artifact where a human has to approve. Then the executor uses the tools to complete the plan. The last step is the reflector which asks do I need to re-plan, or was the task completed.

1 System Design

The agent has three main nodes that communicate through a shared state dictionary (`AgentState`):

- **Planner** — lists the files in the target folder, reads content previews for files with ambiguous names, and asks the LLM for a `{filename to folder}` mapping. Python code converts that mapping into a step list.
- **Executor** — runs one step per graph invocation (create folder, move file, rename file). Supports dry-run mode (shows plan only) and safe mode (asks before each destructive step).
- **Reflector** — only fires when a step fails. It auto-skips errors that will never recover (e.g., file not found), and for everything else asks the LLM whether to retry, skip, or replan.
- **Human In The Loop (HITL)** — prompts the user to approve the plan before proceeding to the execution phase.

The graph loops the Executor via a conditional edge and not a Python for-loop, so the Reflector can intercept failures between steps. A 400-second wall-clock timeout prevents the agent from running indefinitely on large folders.

PLAN REVIEW

Step	Description	Tool
1	Create Finance folder	create_folder
2	Rename file1.txt to invoice_acme_feb2026.txt	rename_file
3	Move invoice_acme_feb2026.txt to Finance folder	move_file
4	Create Personal folder	create_folder
5	Rename file2.txt to book_reading_log_2026.txt	rename_file
6	Move book_reading_log_2026.txt to Personal folder	move_file
7	Create Health folder	create_folder
8	Rename file3.txt to gym_log_mar2026.txt	rename_file
9	Move gym_log_mar2026.txt to Health folder	move_file
10	Create Data folder	create_folder
11	Move file10.csv to Data folder	move_file
12	Move file9.csv to Data folder	move_file
13	Create Personal folder	create_folder
14	Rename file6.jpg to silver_smartphone_closeup.jpg	rename_file
15	Move silver_smartphone_closeup.jpg to Personal folder	move_file
16	Create Food folder	create_folder
17	Rename file7.jpg to black_coffee_latte_art.jpg	rename_file
18	Move black_coffee_latte_art.jpg to Food folder	move_file
19	Create Nature folder	create_folder
20	Rename file8.jpg to evergreen_forest_scenery.jpg	rename_file
21	Move evergreen_forest_scenery.jpg to Nature folder	move_file
22	Move file1.jpg to Personal folder	move_file
23	Move file2.jpg to Personal folder	move_file

Figure 3: The plan presented to the user before execution. The user can approve, reject, or switch to dry-run mode.

1.1 What I Built On

Table 1: External Libraries and Frameworks

Component	Library	What it does in NeatAgent
Agent framework	LangGraph [3]	Wires the Planner/Executor/Reflector nodes together
LLM serving	Ollama	Runs quantized LLMs locally via REST API
LLM client	langchain-ollama	`ChatOllama` used in Planner and Reflector
PDF reading	pdfplumber	Extracts text from PDFs for content-based classification
DOCX reading	python-docx	Extracts text from Word documents
Terminal UI	rich	Colored plan table, progress display, summary
Interactive prompts	questionary	Folder picker and run-option checkboxes
Models used		
Primary	`qwen2.5-coder:14b` (Alibaba)	Best JSON output reliability
Mid-size	`qwen3:8b` (Alibaba)	Faster, slightly lower accuracy
Smallest	`llama3.2:3b` (Meta)	Fastest, lowest accuracy
Vision	`qwen3-vl:8b` (Alibaba)	Describes images for classification

The PLAN-AND-ACT pattern is drawn from Erdogan et al. [1]. The Reflector is modeled after the Reflexion framework [2], where an agent reflects on its own failures before deciding what to do next.

2 What's Novel

2.1 Mapping-Based Planner

The first version of the planner asked the LLM to generate a full list of action steps: move this file here, rename that one, etc. This turned out to be a disaster — on a 10-file folder the model produced 63 steps, 40 of which failed because the model had invented filenames like `file11.pdf` through `file52.kt` that didn't exist. The model would drift after covering the real files and keep generating steps for imaginary ones.

The fix was to change what the LLM is responsible for. Instead of generating steps, it now outputs a simple mapping:

```
{
  "file1.txt": { "folder": "Invoices", "rename": null },
  "file4.pdf": { "folder": "Invoices", "rename": null },
  "file6.jpg": { "folder": "Photos", "rename": null }
}
```

The keys come from the real file listing, so the model can't invent a key that doesn't exist in its input. Python then generates all the steps deterministically. Any key that doesn't match a real file is filtered out before any steps are built. This made hallucination structurally impossible rather than just something we hoped the model would avoid.

2.2 Content-Aware Classification

For files with generic names, the filename alone gives no signal. `file4.pdf` could be anything. NeatAgent solves this by reading a content preview before calling the planner: text files and PDFs are read with pdfplumber, DOCX files with python-docx, and images are described by a vision model. The preview is injected into the planner prompt as a `CONTENT:` hint. This gave a roughly 20 percentage point improvement in placement accuracy for ambiguous files.

2.3 Fully Local, Including the Evaluator

Everything runs locally — not just the agent, but also the LLM-as-judge evaluator. The judge is just another Ollama model that receives the resulting folder structure and ground truth, and returns a 1–10 score with reasoning. No API key needed at any point.

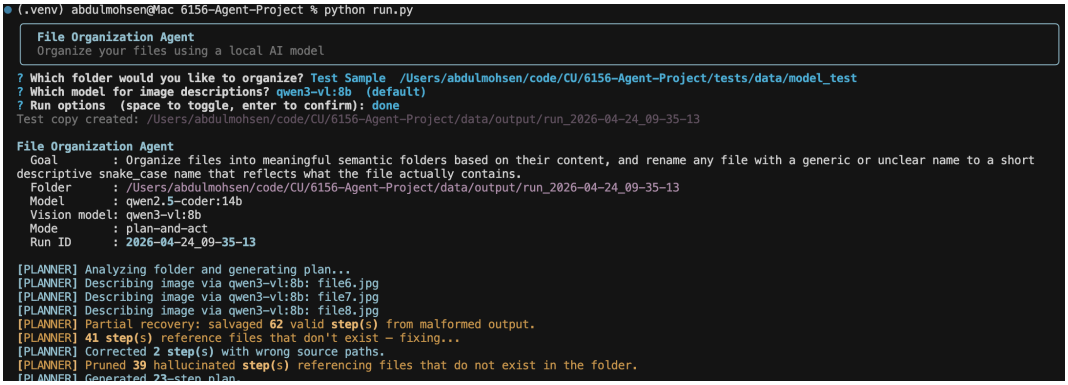

Figure 4: The interactive setup wizard launched on first run via `python run.py`

3 Research Questions

3.1 RQ1: Can local LLMs organize files without cloud APIs?

Motivation. Most AI agent systems require a cloud API (GPT-4, Claude). The question is whether a quantized local model can do this well enough to be useful.

Results. Using `qwen2.5-coder:14b` on the 10-file `model_test` dataset, the agent achieved 100% coverage (every file was moved) and around 60% placement accuracy (6 out of 10 files in the correct semantic folder). The agent correctly grouped invoices, photos, and CSVs. The main misclassifications involved files where the content could reasonably belong in more than one category.

Answer. Yes. A 14B local model gets the job done without cloud dependency and produces results a typical user would find useful.

3.2 RQ2 (Ablation): Which parts of the pipeline actually matter?

I wanted to understand which components were doing the real work versus which ones were just overhead. I tested four conditions:

1. **Step-based planner (original)** — LLM generates executable steps directly, no content preview, no reflector

2. **Mapping planner, no content preview** — LLM sees filenames and sizes only

3. **Mapping planner + content preview, no reflector** — reflector disabled, failed steps auto-skipped

4. **Full system** — mapping planner + content preview + reflector

Table 2: Ablation Results — `model_test` dataset, `qwen2.5-coder:14b`

Condition	Steps	Failures	Hallucinated Files	Placement Acc.
(1) Step-based (baseline)	63	40	∼42	unusable
(2) Mapping, no preview	13	0	0	∼40%
(3) Mapping + preview, no reflector	13	0	0	∼60%
(4) Full system	13	0	0	∼60%

The mapping-based planner was the biggest single change — it took the system from completely broken (condition 1) to functional (condition 2). Content previews added about 20 points of placement accuracy for files with ambiguous names. The Reflector didn't change the numbers on the clean `model_test` runs, but it recovered gracefully during development when files were already moved or folders already existed.

3.3 RQ3: How do local models compare to each other and to a frontier model?

To get a more complete picture, I also evaluated Claude Code (claude-sonnet-4-6, Anthropic's frontier model) on the same dataset. Claude Code operated directly on the files rather than through the NeatAgent pipeline, so I built a separate folder-based evaluator (`eval/eval_folder.py`) that scores any organized folder against the ground truth without requiring a manifest. Claude Code also renamed every file to a descriptive name, so I recorded the rename mapping and passed it to the evaluator.

Table 3: `model_test` results on NeatAgent (three local models) vs. Claude Code

System	Model	Coverage	Placement Acc.	Renamed files	Time
NeatAgent	`qwen2.5-coder:14b`	100%	60%	yes	∼90s
NeatAgent	`qwen3:8b`	100%	50%	yes	∼70s
NeatAgent	`llama3.2:3b`	100%	40%	yes	∼40s
Claude Code	claude-sonnet-4-6	100%	**80%**	yes	interactive

Claude Code achieved 80% placement accuracy — the highest of all systems tested — and also produced the most descriptive renames (e.g., `file4.pdf` became `cloud_services_invoice_2026_0412.pdf`). The two misses were `file3.txt` (gym log) and `file5.pdf` (health report), both placed in `personal/` rather than a dedicated `health/` folder. Claude Code chose to merge health and personal into one broader category, which is a reasonable organizational choice even if it doesn't match the ground truth exactly.

All four systems achieved 100% coverage. The gap between the best local model (60%) and Claude Code (80%) is meaningful but not as large as one might expect. NeatAgent with `qwen2.5-coder:14b` closes about two-thirds of the gap while running entirely offline and for free.

Model	Coverage	Placement	Judge	Duration
qwen2.5-coder:14b	100%	60%	5/10	~62s
qwen3:8b	100%	50%	4/10	~38s
llama3.2:3b	100%	60%	5/10	~22s

Figure 5: Placement accuracy and inference time across three model sizes

4 Value to Users

NeatAgent is for anyone who accumulates disorganized files and doesn't want to manually sort them. The privacy angle is important: if you work with sensitive documents — contracts, medical files, financial records — you probably don't want to upload them to a cloud API to classify them. NeatAgent keeps everything on your machine.

The setup is designed to be as friction-free as possible. Running `python run.py` handles virtual environment creation, dependency installation, and first-time Ollama setup automatically. After that, you pick a folder from a menu and the agent does the rest.

Availability:

- Open-source: `https://github.com/Abdulmohseng/6156-Agent-Project`
- Works on macOS, Linux, and Windows (Python 3.10+)
- Free — no API keys, no subscriptions
- Works offline after models are downloaded
- Hardware: 8–16 GB RAM is enough for the 3B–8B models; 14B needs about 10 GB

5 Evaluation and Metrics

5.1 The Dataset I Created

I built a 10-file dataset (`model_test`) specifically for this evaluation. The goal was to make classification hard: every file has a generic name (file1.txt through file10.csv), so the agent *has* to read the content to know where to put it. The files cover five categories:

- **Invoices** — `file1.txt` (text invoice), `file4.pdf` (PDF invoice)
- **Health** — `file3.txt` (gym log), `file5.pdf` (annual health report)
- **Personal** — `file2.txt` (book reading log)
- **Photos** — `file6.jpg`, `file7.jpg`, `file8.jpg`
- **Data** — `file9.csv` (sensor readings), `file10.csv` (sales data)

Each file has a ground truth label with a primary expected folder and a list of aliases — folder names the agent might reasonably use that should still count as correct. For example, an invoice file accepts "Invoices", "Finance", "Bills", and "billing" as all correct. I also have a larger dataset (`sample_flat`, 24 files) with a mix of generic and descriptive filenames.

5.2 Evaluation Pipeline

```
(.venv) abdulmohsen@Mac 6156-Agent-Project % python3 eval/runner.py --existing-run 2026-04-24_10-41-50 --dataset model_test
  Judging with qwen2.5-coder:14b...
Evaluated existing run: /Users/abdulmohsen/code/CU/6156-Agent-Project/eval/results/run_2026-04-24_10-41-50_qwen2.5-coder_14b.json
```

File Organization Agent — Evaluation Results

Model	Coverage	Placement Acc.	Judge Score	Folder Quality	Files Moved	Replans	Duration (s)	Run ID
qwen2.5-coder:14b	100%	80%	7/10	good	10	0	63.6	2026-04-24_10-41-50

Per-File Placement — qwen2.5-coder:14b (2026-04-24_10-41-50)

Original File	Expected Folder	Actual Folder	Placed?	Match?
file1.txt	invoices	Finance	Yes	Yes
file2.txt	personal_logs	Personal	Yes	Yes
file3.txt	health	Health	Yes	Yes
file4.pdf	invoices	Finance	Yes	Yes
file5.pdf	health	Health	Yes	Yes
file6.jpg	photos	Personal	Yes	No
file7.jpg	photos	Food	Yes	No
file8.jpg	photos	Nature	Yes	Yes
file9.csv	datasets	Data	Yes	Yes
file10.csv	datasets	Data	Yes	Yes

Judge reasoning: The agent has organized most files correctly, but there are some misclassifications and missed files.
Weaknesses: File3.txt is incorrectly placed in the 'Health' folder instead of 'health', Files related to photos (file6.jpg, file7.jpg, file8.jpg) are not grouped together as expected, Missing folders for 'datasets' and 'personal_logs'

Comparison saved: /Users/abdulmohsen/code/CU/6156-Agent-Project/eval/results/comparison_2026-04-24_10-43-32.json

Figure 6: Evaluation pipeline output that cross validates with ground truth data

The pipeline has four parts:

1. `runner.py` — runs the agent as a subprocess with auto-approval, then finds the manifest it wrote to `~/.file-agent/runs/`

2. `compare.py` — walks the execution trace to find where each file ended up, including through rename chains

3. `judge.py` — sends the result to a local Ollama model and asks for a 1–10 score with reasoning

4. `report.py` — prints a colored terminal table and saves JSON

5.3 Metrics

Coverage — what fraction of files were moved at all.

Placement Accuracy — what fraction landed in the right folder (exact or alias match).

LLM-as-Judge Score (1–10) — a local Ollama model sees the organized folder structure and the ground truth, and scores the overall quality with reasoning. Scores of 4–5/10 came back consistently, which was lower than the structural metrics suggested. After looking at the reasoning, the judge was penalizing folder names like "Invoices" when it would have preferred "Finance" — even though both are correct. This highlights that file organization is inherently subjective.

6 Deliverables

Repository: `https://github.com/Abdulmohseng/6156-Agent-Project`

Table 4: Project Deliverables

Deliverable	Location
Project proposal	`docs/project_proposal.pdf`
Progress report	`docs/NeatAgent.pdf`
Final report	`docs/final_report.md`
This paper	`docs/paper.tex`
Agent source code	`file-agent/`
Entry point	`run.py`
First-time setup wizard	`setup/`
Evaluation pipeline	`eval/`
Ground truth labels	`eval/ground_truth/`
Test datasets	`tests/data/model_test/`, `eval/datasets/sample_flat/`
Evaluation run results	`eval/results/`
Build & run instructions	`README.md`

7 Self-Evaluation

This was an individual project. I built the full system from scratch: the LangGraph state machine, the planner rewrite (twice — once step-based, once mapping-based), the executor with dry-run and safe mode, the reflector, five filesystem tools, the evaluation pipeline, two labeled datasets, the setup wizard, and the documentation.

7.1 What I Found Hard

The hallucination problem was the biggest challenge and took the longest to figure out. The step-based planner seemed to work fine on simple test cases during development, so I only discovered the issue when I ran formal evaluation on a slightly more complex folder. At that point the model was producing 63 steps for 10 files and failing 40 of them. Tracking down why — reading the raw LLM output, understanding the context drift, realizing the model was continuing to generate after it ran out of real files — took a while. The fix (changing the output format) was simple once I understood the problem, but getting there wasn't.

Evaluation design was also harder than I expected. Defining "correct" for a subjective task like file organization requires a lot of judgment calls. Building the alias system helped, but it's still imperfect — a real evaluation would benefit from multiple annotators comparing notes.

7.2 Planned but Not Completed

Research Question 4 was originally scoped as a user acceptance study: recruit participants, have them run NeatAgent on their own folders, and collect ratings on whether they trusted and would adopt the AI-generated organization. This never happened. The main obstacle was hardware and finding participants. Most people I asked had a machine that was either too slow to run a 14B model locally or couldn't install Ollama at all.

In my progress report I mentioned that I was going to use Gemma4 from google; their new open-source model, however the model was not able to perform at all. I decided to omit it from the project and only use the models I already built the agent with.

7.3 What I Learned

The most lasting thing I took from this project is that output format is more important than model capability for agentic tasks. I spent time trying to fix the hallucination problem through better prompting with more explicit instructions, stronger warnings, different phrasing. None of it helped. Changing the structure of what the model was asked to output fixed it immediately. When you can constrain the output schema so that the error can't be represented, you've solved the problem in a way that's reliable regardless of which model you're using.

The other thing I learned is that small local models are more capable than I expected for tasks with a well-defined output format. A 3B model running on a laptop can organize files semantically. That's genuinely useful and opens up automation for people who can't or won't use cloud APIs. I believe that with new powerful chips being produced and models being accessible we will start to see more people building and automating with local models.

7.4 Course Feedback

I truly enjoyed this course to the fullest! Having the freedom to explore topics that are interesting to me, writing about them, and building with them is very rewarding. I would absolutely recommend this course to anyone who wants to explore a topic and enjoys learning from others. Reading the papers weekly in the first half of the term is really a skill that I will carry with me into industry and research.

References

[1] Lutfi Eren Erdogan et al. PLAN-AND-ACT: Improving Planning of Agents for Long-Horizon Tasks. *arXiv preprint arXiv:2503.09572*, 2025. https://arxiv.org/pdf/2503.09572

[2] Anthropic. Building Effective Agents. Blog, 2024. https://www.anthropic.com/engineering/building-effective-agents

[3] LangChain. LangGraph: Build Stateful, Multi-Actor Applications with LLMs. https://github.com/langchain-ai/langgraph, 2024.

[4] Taicheng Guo et al. Large Language Model based Multi-Agents: A Survey of Progress and Challenges. *arXiv preprint arXiv:2402.01680*, 2024. https://arxiv.org/pdf/2402.01680

[5] John Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *NeurIPS*, 2024. https://arxiv.org/abs/2405.15793

Appendix

.1 Planner Prompt

```
You are a file organization assistant. Given a list of files,
output a JSON object that maps each filename to its destination.

Output schema -- one key per file, exactly:
{
  "<exact_filename_from_listing>": {
    "folder": "<SemanticFolderName>",
    "rename": "<new_stem_no_extension_or_null>"
  }
}

Rules:
- Output a key for EVERY file. No more, no less. Never invent filenames.
- folder: short, meaningful name (Finance, Health, Work, Photos, etc.)
- rename: snake_case stem only -- set only when filename is generic.
- Use CONTENT and IMAGE hints to classify accurately.
Output ONLY valid JSON. No markdown, no explanation.
```

.2 Ground Truth Schema

```
"file4.pdf": {
  "expected_folder": "invoices",
  "folder_aliases": ["Finance", "Bills", "Receipts"],
  "expected_name": null,
  "content_hint": "Invoice from Acme Corp, March 2026"
}
```